

# POO en Unity: estructura + ejercicio practico

Vamos a ampliar la propuesta del Ghost DJ para que no se quede en "copiar tres scripts". La meta es montar una escena jugable minima, conectar referencias entre objetos, probar el flujo en Play Mode y tener tareas de ampliacion.

## Resultado que debes conseguir

Al pulsar ESPACIO: el GhostDJ intenta lanzar un BeatSpell. Si todo esta conectado, el objetivo recibe 1 golpe, el contador interno aumenta y el coste de energia se aplica. Al llegar a 3 golpes, el DanceTarget cambia de color y gira durante unos segundos.

## 1. El mini juego

Imagina una sala de un juego musical. Tu personaje fantasma tiene un BeatSpell equipado y delante hay un objetivo que responde al ritmo. No hay inventario, enemigos ni fisica: solo objetos que colaboran entre si.

### La relacion central es:

GhostDJ --> BeatSpell --> DanceTarget

Esto permite practicar una idea muy importante de POO en Unity: un objeto puede guardar una referencia a otro objeto y pedirle que haga su responsabilidad, en lugar de poner todo el comportamiento dentro de una unica clase.

## 2. Tu objetivo como estudiante

| Paso | Objetivo                             | Que practicas                           |
|------|--------------------------------------|-----------------------------------------|
| 1    | Crear los 3 GameObjects              | Componentes y jerarquia basica          |
| 2    | Asignar las referencias en Inspector | Objetos de tipo BeatSpell y DanceTarget |
| 3    | Probar ESPACIO                       | Metodos y flujo entre clases            |
| 4    | Llegar a 3 golpes                    | Estado privado y reglas internas        |
| 5    | Completar ampliaciones               | Diseño y responsabilidad de clases      |

## 3. Montaje de la escena desde cero

Haz esta parte exactamente una vez y luego usa la escena como laboratorio para las ampliaciones.

### 3.1 Jerarquia

```
SampleScene
|- GhostDJ
|- BeatSpell
|- DanceTarget
```

#### Importante

Para este ejercicio no necesitas hacerlos hijos unos de otros. Son tres objetos independientes que se comunican mediante referencias. Esto hace visible la diferencia entre jerarquia Transform y relacion POO.

### 3.2 Crear los objetos

| GameObject  | Componentes                                       | Transform recomendado |
|-------------|---------------------------------------------------|-----------------------|
| GhostDJ     | GhostDJ + opcionalmente un cubo/esfera para verlo | Position (0, 0, 0)    |
| BeatSpell   | BeatSpell                                         | Position (0, 1.5, 0)  |
| DanceTarget | DanceTarget + Cube o Sphere                       | Position (0, 0, 5)    |

### 3.3 Ajustar el Inspector

| Objeto      | Campo         | Valor inicial            | Por que                       |
|-------------|---------------|--------------------------|-------------------------------|
| GhostDJ     | Selected Beat | BeatSpell del proyecto   | Es la referencia POO          |
| GhostDJ     | Energy        | 30                       | Cada golpe cuesta 10          |
| BeatSpell   | Target        | DanceTarget del proyecto | Indica a quien afecta         |
| BeatSpell   | Power         | 1                        | Cada lanzamiento suma 1 golpe |
| DanceTarget | Required Hits | 3                        | Con 3 golpes se activa        |
| DanceTarget | Dance Time    | 2                        | Dura 2 segundos la reaccion   |

## 4. Los 3 scripts

Copia cada script en un archivo con el mismo nombre de la clase. El control de prueba usa el New Input System para poder lanzar el beat con ESPACIO sin crear acciones ni una UI adicional. Si tu proyecto esta en Input Manager (Old), cambia Active Input Handling a Input System Package (New) o Both.

### GhostDJ.cs

```
using UnityEngine;
using UnityEngine.InputSystem;

public class GhostDJ : MonoBehaviour
{
    [SerializeField] private BeatSpell selectedBeat;
    public float energy = 30f;
    private int combo;

    private void Update()
    {
        if (Keyboard.current != null && Keyboard.current.spaceKey.wasPressedThisFrame)
            PerformBeat();
    }

    public void PerformBeat()
    {
        if (selectedBeat == null)
        {
            Debug.LogWarning("GhostDJ: falta asignar Selected Beat.");
            return;
        }

        if (energy < 10f)
        {
            Debug.Log("GhostDJ: no hay energia suficiente.");
            return;
        }

        if (selectedBeat.Cast())
        {
            energy -= 10f;
            combo++;
            Debug.Log($"Beat lanzado. Combo: {combo} | Energia: {energy}");
        }
    }

    private void OnGUI()
    {
        GUI.Label(new Rect(20, 20, 380, 25),
            $"ESPACIO = lanzar beat | Energia: {energy:0} | Combo: {combo}");
    }
}
```

### BeatSpell.cs

```
using UnityEngine;

public class BeatSpell : MonoBehaviour
{
    [SerializeField] private DanceTarget target;
    public int power = 1;
    private float cooldown;

    private void Update()
    {
        cooldown = Mathf.Max(0f, cooldown - Time.deltaTime);
    }

    public bool Cast()
    {
        if (target == null)
        {
            Debug.LogWarning("BeatSpell: falta asignar Target.");
            return false;
        }

        if (cooldown > 0f)
        {
            Debug.Log("BeatSpell: en cooldown.");
            return false;
        }

        target.ReceiveBeat(power);
        cooldown = 0.5f;
        return true;
    }
}
```

## DanceTarget.cs

```
using System.Collections;
using UnityEngine;

public class DanceTarget : MonoBehaviour
{
    public int requiredHits = 3;
    public float danceTime = 2f;
    private int hits;

    public void ReceiveBeat(int power)
    {
        hits += power;
        Debug.Log($"DanceTarget: {hits}/{requiredHits} golpes.");

        if (hits >= requiredHits)
        {
            hits = 0;
            StartCoroutine(Dance());
        }
    }

    private IEnumerator Dance()
    {
        Renderer r = GetComponent<Renderer>();
        Color original = r != null ? r.material.color : Color.white;

        if (r != null)
            r.material.color = Color.cyan;

        float elapsed = 0f;
        while (elapsed < danceTime)
        {
            transform.Rotate(0f, 240f * Time.deltaTime, 0f);
            elapsed += Time.deltaTime;
            yield return null;
        }

        if (r != null)
            r.material.color = original;
    }
}
```

## 5. Como conectar las referencias

Aqui esta la parte que suele faltar en un ejercicio de POO: el codigo puede estar perfecto y, aun asi, no hacer nada si las referencias no estan conectadas en el Inspector.

### Paso 1 - GhostDJ

Selecciona GhostDJ. En el componente GhostDJ veras el campo Selected Beat. Arrastra el GameObject BeatSpell desde la Hierarchy hasta ese campo.

#### Que acaba de ocurrir?

La variable selectedBeat es de tipo BeatSpell. Ahora contiene una referencia real a la instancia BeatSpell de tu escena. GhostDJ puede llamar a selectedBeat.Cast().

### Paso 2 - BeatSpell

Selecciona BeatSpell. En el componente BeatSpell, busca Target y arrastra DanceTarget desde la Hierarchy hasta ese campo.

#### Segunda relacion

BeatSpell ahora tiene una referencia a un objeto de tipo DanceTarget. Por eso puede ejecutar target.ReceiveBeat(power) sin conocer como esta implementada la reaccion del objetivo.

### Paso 3 - Comprueba el grafo

```
GhostDJ.selectedBeat
-> BeatSpell
    -> BeatSpell.target
        -> DanceTarget
```

Si uno de los dos enlaces esta vacio, el sistema no puede completar la cadena.

## 6. Primera prueba en Play Mode

- 1 Pulsa Play.
- 2 Mira la consola: no deberia aparecer ningun warning de referencia vacia.
- 3 Pulsa ESPACIO una vez. Deberias ver "1/3 golpes" en Console.
- 4 Espera al menos 0.5 s y pulsa ESPACIO otra vez.
- 5 Repite hasta llegar a 3/3.
- 6 El DanceTarget deberia ponerse cyan y girar durante 2 segundos.
- 7 La energia del GhostDJ deberia bajar 10 por lanzamiento. El Combo deberia subir.

### Prueba correcta

Con una escena bien conectada, deberias poder explicar el recorrido completo: Space -> GhostDJ.PerformBeat() -> BeatSpell.Cast() -> DanceTarget.ReceiveBeat() -> DanceTarget.Dance().

### Antes de avanzar

Escribe en un papel el flujo de objetos sin mirar el codigo. Luego identifica donde vive cada dato: energy en GhostDJ, cooldown en BeatSpell y hits en DanceTarget. Esta comprobacion separa la comprension de la simple copia.

## 7. Haz que el ejercicio sea tuyo

Cuando la prueba basica funcione, deja de mirar el codigo y completa estas pequeñas tareas. No cambies la cantidad maxima de clases: la idea es mejorar el comportamiento manteniendo responsabilidades claras.

### Nivel 1 - Cambia reglas

Modifica solo valores del Inspector y comprueba que puedes alterar el juego sin tocar el codigo.

```
DanceTarget.requiredHits = 5  
BeatSpell.power = 2  
GhostDJ.energy = 50
```

Preguntas: ¿cuantos lanzamientos necesitas ahora? ¿La energia alcanza? ¿Que ocurre si Power vale 3?

### Nivel 2 - Personaliza la recompensa visual

En DanceTarget.Dance(), cambia el color y la velocidad de giro. El objetivo debe seguir siendo responsable de su propia reaccion.

```
r.material.color = Color.magenta;  
transform.Rotate(0f, 420f * Time.deltaTime, 0f);
```

### Nivel 3 - Evita lanzar cuando no queda energia

Ya existe la regla, pero ahora comprueba la experiencia de juego: deja que la energia llegue a 0 y pulsa ESPACIO. El sistema debe avisarte y no lanzar el beat.

### Nivel 4 - Provoca errores a proposito

Esto es parte del aprendizaje. Borra temporalmente Selected Beat o Target en el Inspector y vuelve a probar. Lee el warning y decide que referencia falta.

#### Meta

Debes ser capaz de diagnosticar el problema mirando primero el flujo de referencias y luego la Console. No empieces modificando codigo si el Inspector tiene una referencia vacia.

## 8. Reto de ampliacion: dos objetivos

Crea un segundo conjunto BeatSpell + DanceTarget. No crees nuevas clases. El reto consiste en pensar como permitiras que GhostDJ cambie de BeatSpell sin convertir la clase en una coleccion desordenada de referencias.

Una posible direccion es introducir una lista o un sistema de seleccion mas adelante. En esta fase no hace falta resolverlo con una arquitectura compleja: primero dibuja el diseño en papel y marca que objeto conoce a cual.

## 9. Que debes aprender realmente aqui

| Concepto                   | En el ejercicio                                          | Compruebalo preguntandote...                                         |
|----------------------------|----------------------------------------------------------|----------------------------------------------------------------------|
| Clase                      | GhostDJ, BeatSpell, DanceTarget                          | ¿Que responsabilidad tiene cada una?                                 |
| Instancia                  | Los GameObjects que llevan esos scripts                  | ¿Donde existe realmente el objeto?                                   |
| Referencia                 | selectedBeat y target                                    | ¿Que objeto puedo alcanzar desde esta variable?                      |
| Encapsulacion              | combo, cooldown y hits son privados                      | ¿Quien puede cambiar este estado?                                    |
| SerializeField             | Campos privados editables en Inspector                   | ¿Puedo configurarlo sin hacerlo publico?                             |
| Metodos                    | PerformBeat, Cast, ReceiveBeat                           | ¿Quien pide la accion y quien la ejecuta?                            |
| Composicion / colaboracion | GhostDJ utiliza BeatSpell; BeatSpell utiliza DanceTarget | ¿Estoy repartiendo responsabilidades o metiendolo todo en una clase? |

## 10. Checklist antes de darlo por terminado

- ☐ Los 3 scripts compilan sin errores.
- ☐ GhostDJ tiene Selected Beat asignado.
- ☐ BeatSpell tiene Target asignado.
- ☐ ESPACIO llama a PerformBeat.
- ☐ El cooldown impide lanzar continuamente.
- ☐ La energia baja solo cuando el BeatSpell se lanza correctamente.
- ☐ DanceTarget cuenta golpes internamente.
- ☐ Al llegar al requisito, el objetivo reacciona visualmente.
- ☐ Puedes explicar que dato es publico, cual es privado y por que.
- ☐ Puedes dibujar las referencias sin mirar el codigo.

### Siguiente paso recomendado

Cuando termines este ejercicio, el siguiente salto de POO seria introducir interfaces o ScriptableObjects. No los metas todavia en este ejercicio: primero consigue entender perfectamente la cadena de referencias entre estas tres clases.